

Lecture 010

Unsupervised learning, dimensionality reduction,
and image classification

Edward Rubin

Admin

Today

Topics: We've got a lot (coming to the end)!

- K-means clustering & unsupervised learning;
- PCA and UMAP for dimension reduction;
- why geometry matters for image classification.

Upcoming

Projects due this week and next! (Also: Final exam.)

Readings

- *Today:* ISL Ch. 12 (*Unsupervised Learning*)
- *Next:* NNs and Deep Learning Ch. 1-2; ISL Ch. 10 (*Deep Learning*).

Predicting digits

7	2	4	5	7	9	8	8	9	7	7	2
2	4	4	6	0	6	4	5	9	3	8	8
1	3	6	9	3	0	1	3	1	1	7	7
8	8	7	2	0	7	3	9	5	2	6	6
7	5	4	6	7	4	8	0	7	0	9	5
5	8	4	9	9	6	7	6	4	4	0	5
5	5	7	1	5	1	9	1	6	4	7	6
9	4	6	8	6	0	7	0	2	5	3	5

Predicting digits

Why digits?

Handwritten digits are a useful **machine-vision sandbox**:

- **784 pixels** per image (28×28),
- **10 labels**, so this is multiclass prediction,
- **easy** to **for us** visualize—and an easy task for humans;
- **difficult** to separate **for many ML methods**.

Bonus: Connects **unsup. learning**, **dim. reduction**, **image/multi-class pred.**

We use Kaggle MNIST files: `train.csv` and `test.csv`.

Predicting digits

Image classification (in one slide)

For **MNIST**[†]

- **predictors:** pixel intensities, (each $x_j \in [0-255]$)
- **target** = the digit label, ($y \in \{0, 1, \dots, 9\}$)

⇒ each image is one point in **784-dimensional** predictor space.

[†] MNIST is a classic benchmark for ML. It contains handwritten digits from the US National Institute of Standards and Technology (written by Census employees and HS students).

Predicting digits

Image classification (in one slide)

For **MNIST**[†]

- **predictors:** pixel intensities, (each $x_j \in [0-255]$)
- **target** = the digit label, ($y \in \{0, 1, \dots, 9\}$)

⇒ each image is one point in **784-dimensional** predictor space.

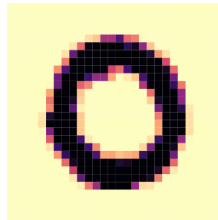
The target is simple. The **geometry** is the hard part.

Why? **Combinations** of pixels matter. (Not so much individual pixels.)

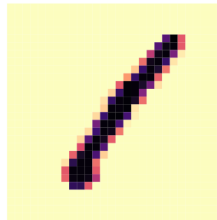
[†] MNIST is a classic benchmark for ML. It contains handwritten digits from the US National Institute of Standards and Technology (written by Census employees and HS students).

What do the digits look like?

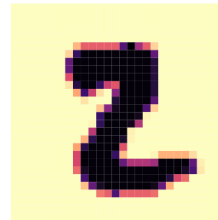
Label = 0



Label = 1



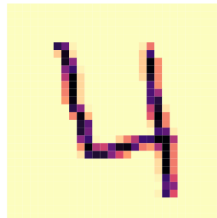
Label = 2



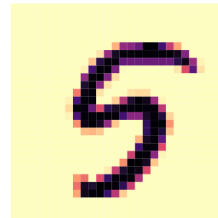
Label = 3



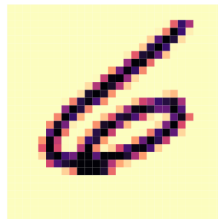
Label = 4



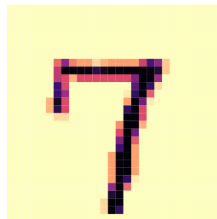
Label = 5



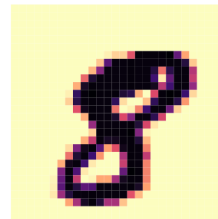
Label = 6



Label = 7



Label = 8



Predicting digits

Kaggle's Setup

As usual, Kaggle gives us two files

- **train.csv** has images and labels
- **test.csv** has images only

Today we use the labeled sample to

1. fit the unsupervised methods,
2. check what structure those methods recover.

Predicting digits

Supervised vs. unsupervised learning

Predicting digits

What's the difference?

Supervised learning

- **train** with X and Y
- **goal:** predict Y
- *ex:* lasso, RFs, SVMs

Unsupervised learning

- **train** with X only
- **goal:** find structure in X
- *ex:* clustering & dim. reduction

Today we start on the **unsupervised** side, but we will keep asking:

Does the structure help prediction?

(In reality, we have a supervised task.)

Predicting digits

K-means clustering

Predicting digits

The general idea

K-means is a simple clustering method groups our data into K groups:

- choose a number of **clusters**, K ; *clusters* = groups of similar obs.
- assign each observation to (exactly) one cluster k ; "nearest" cluster
- summarize each cluster by its **centroid**. center cluster's obs.

K-means compresses a messy cloud into K **representative centers**.

Predicting digits

The general idea

K-means is a simple clustering method groups our data into K groups:

- choose a number of **clusters**, K ; *clusters* = groups of similar obs.
- assign each observation to (exactly) one cluster k ; "nearest" cluster
- summarize each cluster by its **centroid**. center cluster's obs.

K-means compresses a messy cloud into K **representative centers**.

Now we just need to an algorithm to find the *best* clustering..
which probably means defining *best* (i.e., an **objective function**).

Predicting digits

K-means: Objective function

Let cluster k have centroid μ_k . *(The mean location of its obs. in p dim.)*

K-means chooses clusters $C_1, \dots, C_K$ to minimize **within-cluster variation**

$$\min_{C_1, \dots, C_K} \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

E.g., **within-cluster sum of squares (WCSS)** above.

Interpretation

- reward **tight, compact** clusters, *(short distances add little to the sum)*
- "penalize" observations far from their centroids. *(or "bad" clusters.)*

Predicting digits

K-means: Algorithm

A general algorithm for clustering:

1. pick K initial **centroids**; (*randomly, or with a heuristic*)
 2. **assign** each observation to its nearest centroid; (*Euclidean distance*)
 3. **recompute** the centroids; (*mean of the obs. in each cluster*)
 4. **repeat** until assignments stop changing. (*"convergence"*)
- + Fast and intuitive.
 - Only guarantees (approx.) a local optimum.

Predicting digits

K-means on pixels

Recall: Each image (obs.) is one point in 784 dimensions.

There are **10 labels**. $K = 10$ seems like a natural first guess.

... however, $K = 10$ is a guess; not a theorem.

Q: Why might $K = 10$ be a bad choice?

Predicting digits

Turning Clusters into Predictions

We have **clusters**. (*Groups of "close" images in pixel space.*)

We want **predictions**. (*Predicted digit labels.*)

- Cluster labels, i.e., $1, 2, \dots, K$ are arbitrary. (*they just index the clusters*)
- *Solution:* Relabel each cluster by its **modal label**; (*like RF: majority vote*)

Result: With $K = 10$, this classifier is approx. 54.2% accurate.[†]

[†] This measure of accuracy is still *in-sample*.

Predicting digits

Turning Clusters into Predictions

We have **clusters**. (*Groups of "close" images in pixel space.*)

We want **predictions**. (*Predicted digit labels.*)

- Cluster labels, i.e., $1, 2, \dots, K$ are arbitrary. (*they just index the clusters*)
- *Solution*: Relabel each cluster by its **modal label**; (*like RF: majority vote*)

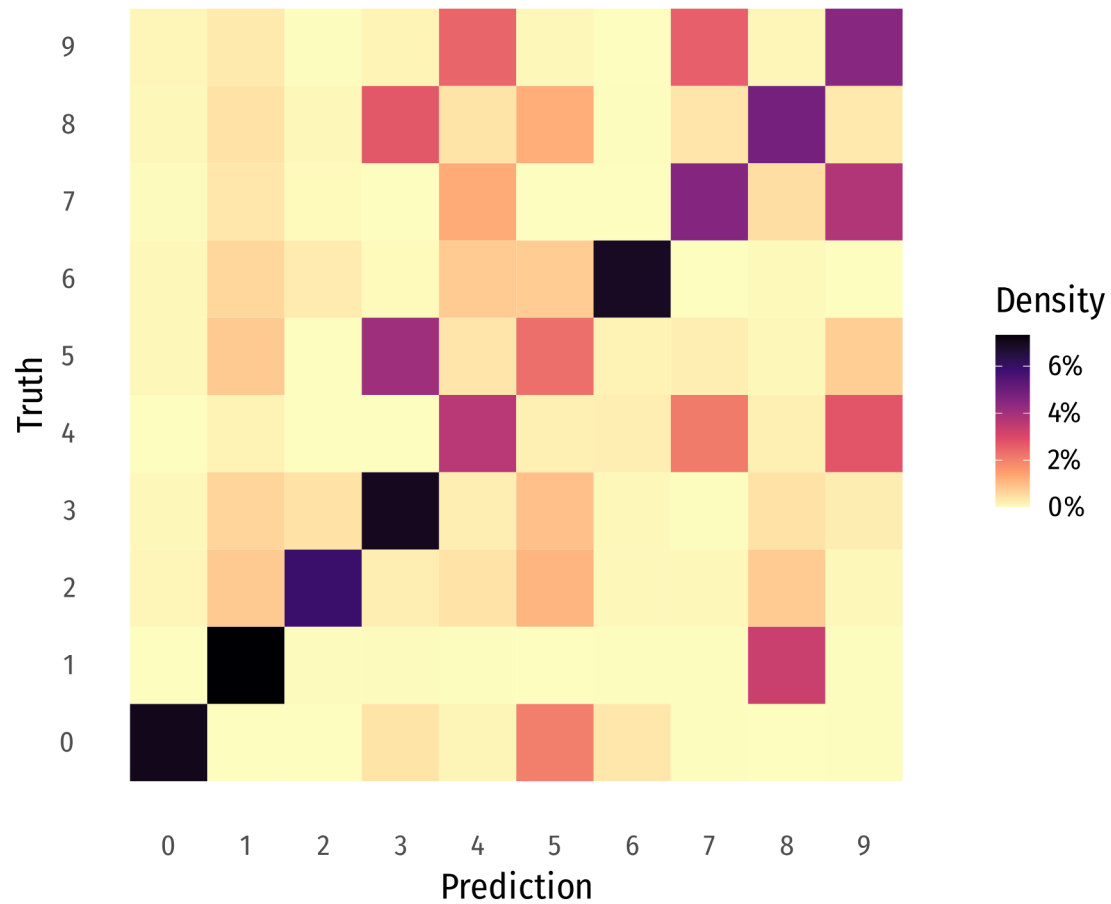
Result: With $K = 10$, this classifier is approx. 54.2% accurate.[†]

Q₁ Is this level of accuracy good?

Q₂ What would random guessing get us?

[†] This measure of accuracy is still *in-sample*.

Confusion matrix, $K = 10$



Predicting digits

Why aren't we doing better?

What is it about the task—and our approach—that isn't working?

We may need more than 10 clusters to capture **within-label variation**

- one **label** can contain several **handwriting styles**;
- the same **digit** can shift **left, right, up, or down**;
- strokes can be **thicker, thinner, or slanted**;
- some characters are **quite ambiguous**.

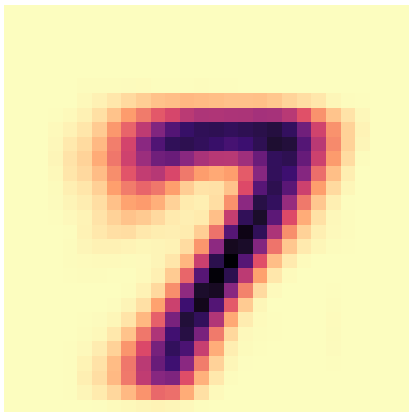
So Euclidean distance in raw pixel space is a rough measure of similarity.

Accuracy by label for $K = 10$ (sensitivity)

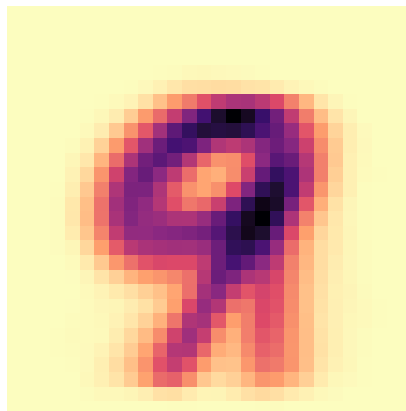
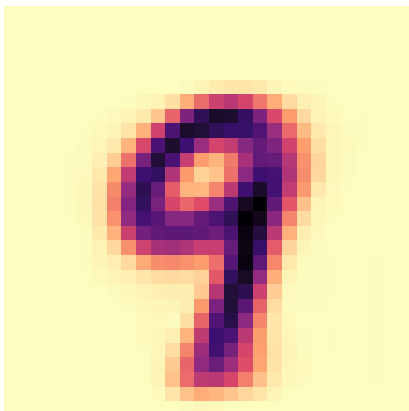
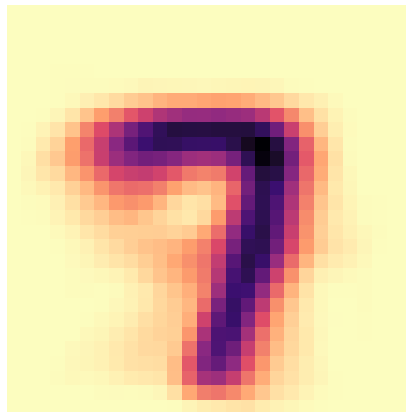
Digit	Accuracy	Count
0	70.2%	604
1	68.0%	647
2	61.2%	582
3	69.1%	608
4	38.9%	560
5	26.1%	536
6	72.4%	576
7	43.3%	631
8	46.0%	632
9	43.3%	624

Separating 7 and 9 is difficult: *Correct* guesses vs. *incorrect* guesses

Correct



Mistakes



Predicting digits

What's K ?

As you might have guessed, **the choice of K** is a big deal for K-means.

- The number of (useful) clusters is usually **unknown**.
- *Common:* The **"elbow"** & downstream **usefulness**. (*marginal gains*)
- The same **hyper-parameter tuning** logic we use throughout the course.

Predicting digits

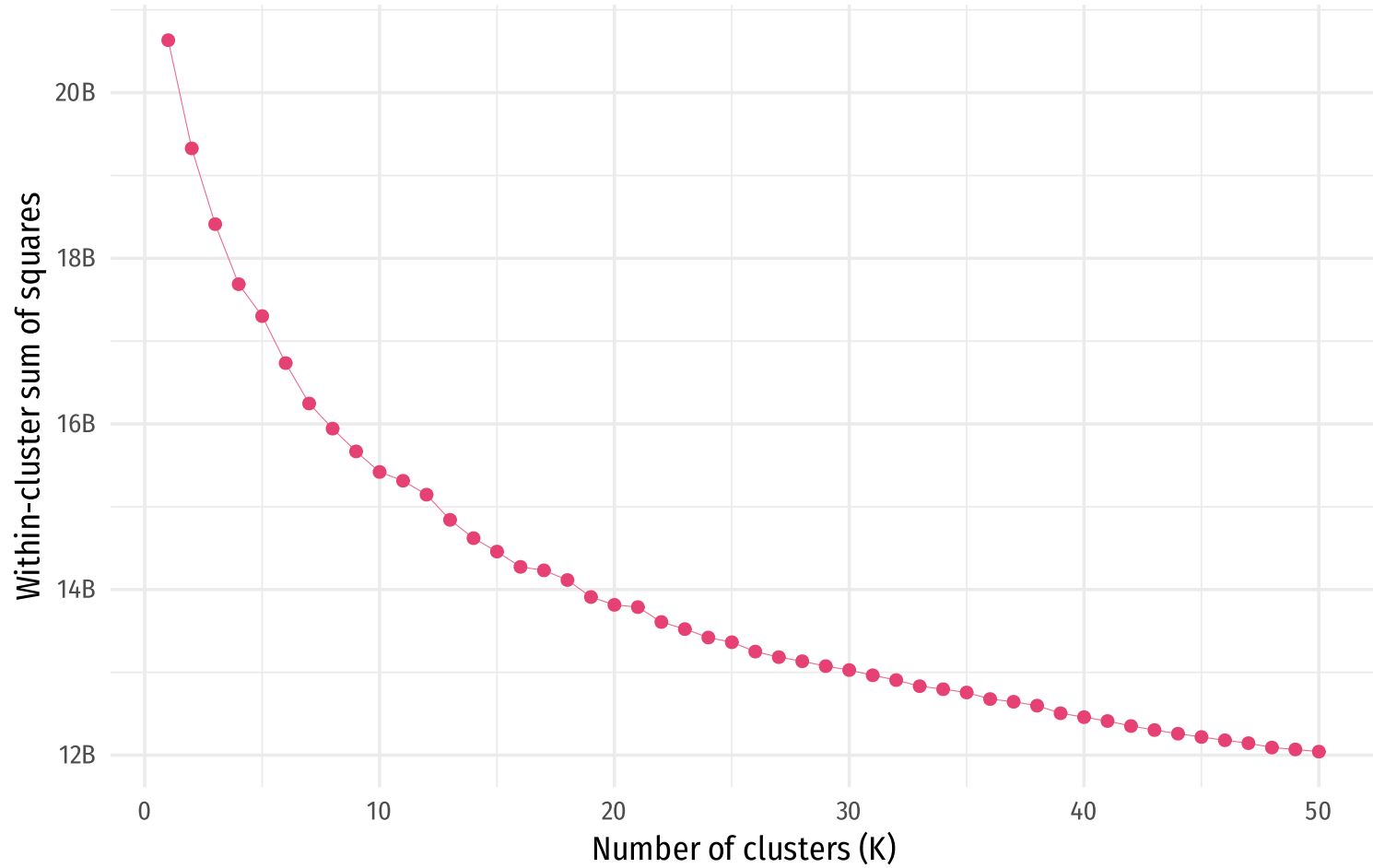
What's K ?

As you might have guessed, **the choice of K** is a big deal for K-means.

- The number of (useful) clusters is usually **unknown**.
- *Common:* The **"elbow"** & downstream **usefulness**. (*marginal gains*)
- The same **hyper-parameter tuning** logic we use throughout the course.

So... we should probably *tune* it.

Finding the elbow. *You see it, right?*



Predicting digits

Elbows and clusters

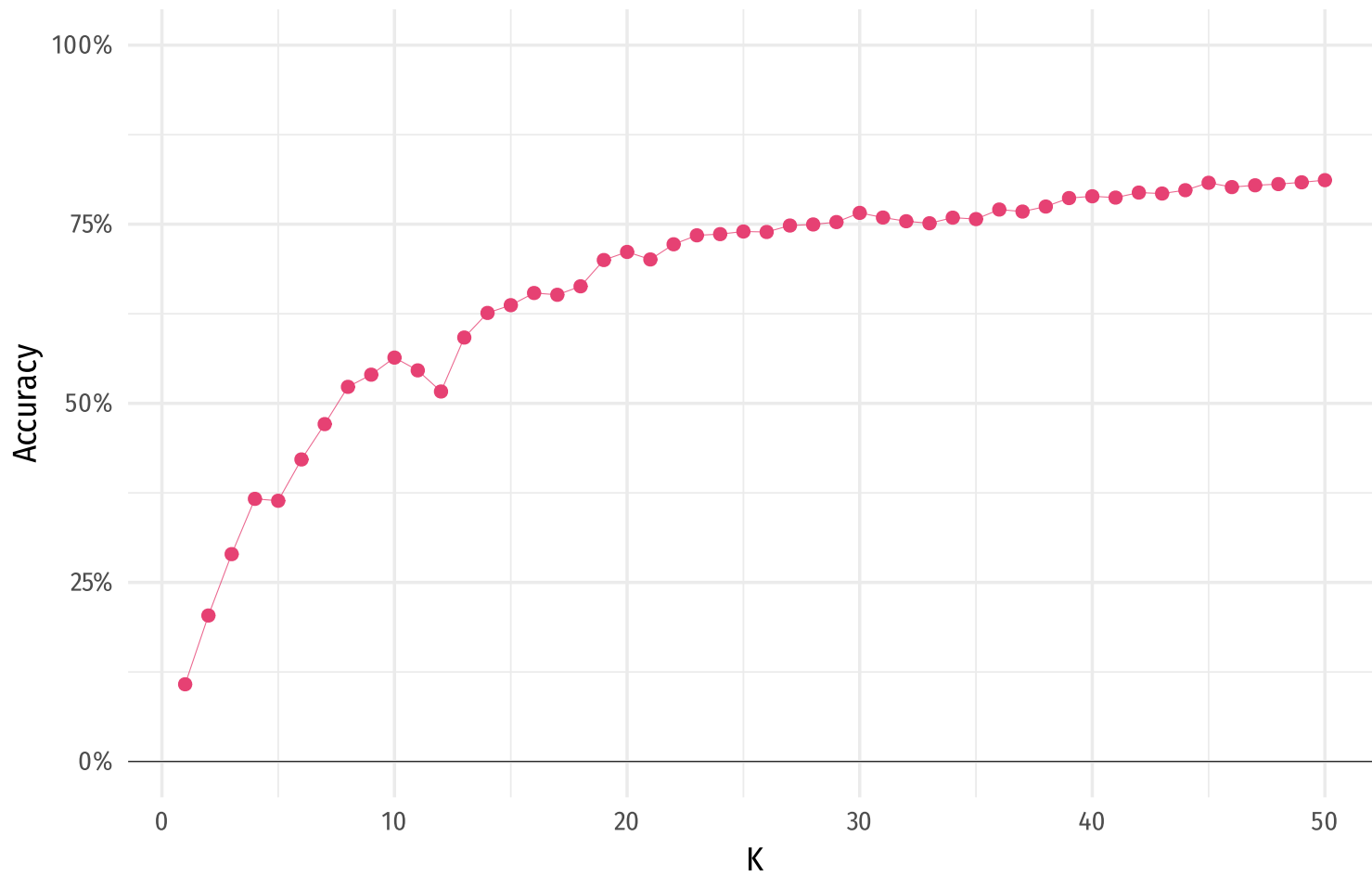
Q What's the *elbow*?

A The bendy part of the curve, of course!

- WCSS mechanically falls as K rises.
- We're looking for the point of **diminishing marginal gains**.
- *Caveat* If the bend is weak, the choice of K stays ambiguous.

Elbows are useful (subjective) heuristics—not *proper* decision rules.

Modal cluster-label accuracy by K . (*Still in-sample.*)



Predicting digits

K -means takeaways

Accuracy depends on K , varies by label, and is *meh*.

- With raw pixels, $K = 10$ gives about 54.2%.
- The best value is around $K = 50$ with 81.2%.
- **Key 10 labels** does *not* imply **10 clusters**.

More clusters can help.

- One true **digit** can split into several **styles**.
- Extra clusters can improve the modal-label benchmark.
(*But can overfit too.*)

Predicting digits

Now with PCA!

Predicting digits

PCA: Intuition

Principal component analysis (PCA) is a **dimension-reduction** method that finds new axes to summarize the data.[†]

Same data; different coordinates.

- The **first** PC explains the **most** variation...
- later PCs explain the most remaining *orthogonal* variation.
- We can use PCA to replace *many* pixel-based features with fewer **linear combinations** of pixels.^{††}

A relatively **small subset** of PCs can give a compact summary of data.

⇒ PCA can **reduce dimensions** while **preserving information**!

[†] PCA is applied in other contexts beyond dimension reduction.

^{††} Linear combinations of pixels are like "superpixels" that capture common patterns.

Predicting digits

PCA: A little math

The first principal component is

$$Z_1 = \phi_{1,1}X_1 + \phi_{2,1}X_2 + \cdots + \phi_{p,1}X_p$$

with analogous definitions for the other $p - 1$ PCs $Z_2, Z_3, \dots, Z_p$.

Don't be overwhelmed by the notation.

Each PC is just a weighted combo. of the original features. *(Just like OLS.)*

Key ideas:

- PCs are **linear combinations** of the original variables,
- PC_i is **orthogonal** to all other PCs,[†]
- we usually **normalize** variables before PCA.

[†] Links: [Nice visualizations](#) plus another [resource](#).

Updating our K-means workflow to include PCA

1. **normalize** the pixels,
2. keep enough PCs to **explain 90% of variance**,
3. **run K-means** in the PC space.

```
# Define a simple PCA recipe
simple_recipe_pca =
  recipe(
    label ~ .,
    data =
      analysis_dt ▷
      select(label, starts_with('pixel'))
  ) ▷
  step_mutate(label = as.factor(label)) ▷
  step_normalize(all_numeric_predictors()) ▷
  step_pca(all_predictors(), threshold = .9) ▷
  step_rm(starts_with('pixel'))
```


Should we have standardized the pixels with plain K -means clustering?

Predicting digits

PCA: What is it good for?

So what does PCA do for us here? Subsetting to the PCs that represent 90% of the variance

- leaves us with **188 PCs**, ($\ll 784$)
- has **45.1%** accuracy with $K = 10$ clusters. ($< 54.2\%$)

Predicting digits

PCA: What is it good for?

So what does PCA do for us here? Subsetting to the PCs that represent 90% of the variance

- leaves us with **188 PCs**, ($\ll 784$)
- has **45.1%** accuracy with $K = 10$ clusters. ($< 54.2\%$)

Wait. So... it's worse?

1. Yes, if we only consider *training accuracy*.
2. Maybe, if we consider the *predictor space* was 24% of the original size.
3. TBD, if we actually check the *test-set accuracy*.

We have a more *compact* representation, but we *did* lose some information.

Predicting digits

What are we doing?

Let's think about what we're actually doing here.

- We are **still running K-means**.
- Only the **coordinates** change with PCA.
- Comparable accuracy with far **fewer dimensions** is a decent trade.

We *could* add more PCs. That road eventually leads to the original space.

Predicting digits

What are we doing?

Let's think about what we're actually doing here.

- We are **still running K-means**.
- Only the **coordinates** change with PCA.
- Comparable accuracy with far **fewer dimensions** is a decent trade.

We *could* add more PCs. That road eventually leads to the original space.

What if we tried a **nonlinear** dimension reduction method instead?

Predicting digits

UMAP No, *UMAP*!

Predicting digits

UMAP: Intuition

UMAP[†] is a popular **nonlinear dimension-reduction method** that can capture more complex structures than PCA.

- **Start:** a high-dim. graph of the data—edges connect nearby obs.;
- **Then:** optimized low-dim. graph that preserves local structure.
- **Nonlinear:** can uncover *curved* structure that PCA misses.

[†] UMAP stands for *Uniform Manifold Approximation and Projection*.
Examples [here](#); paper's repo [here](#).

Predicting digits

UMAP: Intuition

UMAP[†] is a popular **nonlinear dimension-reduction method** that can capture more complex structures than PCA.

- **Start:** a high-dim. graph of the data—edges connect nearby obs.;
- **Then:** optimized low-dim. graph that preserves local structure.
- **Nonlinear:** can uncover *curved* structure that PCA misses.

The axes help with geometry—not direct interpretation

- strong for **visualization**; useful for **clustering**;
- **less interpretable** and more **tuning-sensitive** than PCA.

[†] UMAP stands for *Uniform Manifold Approximation and Projection*.

Examples [here](#); paper's repo [here](#).

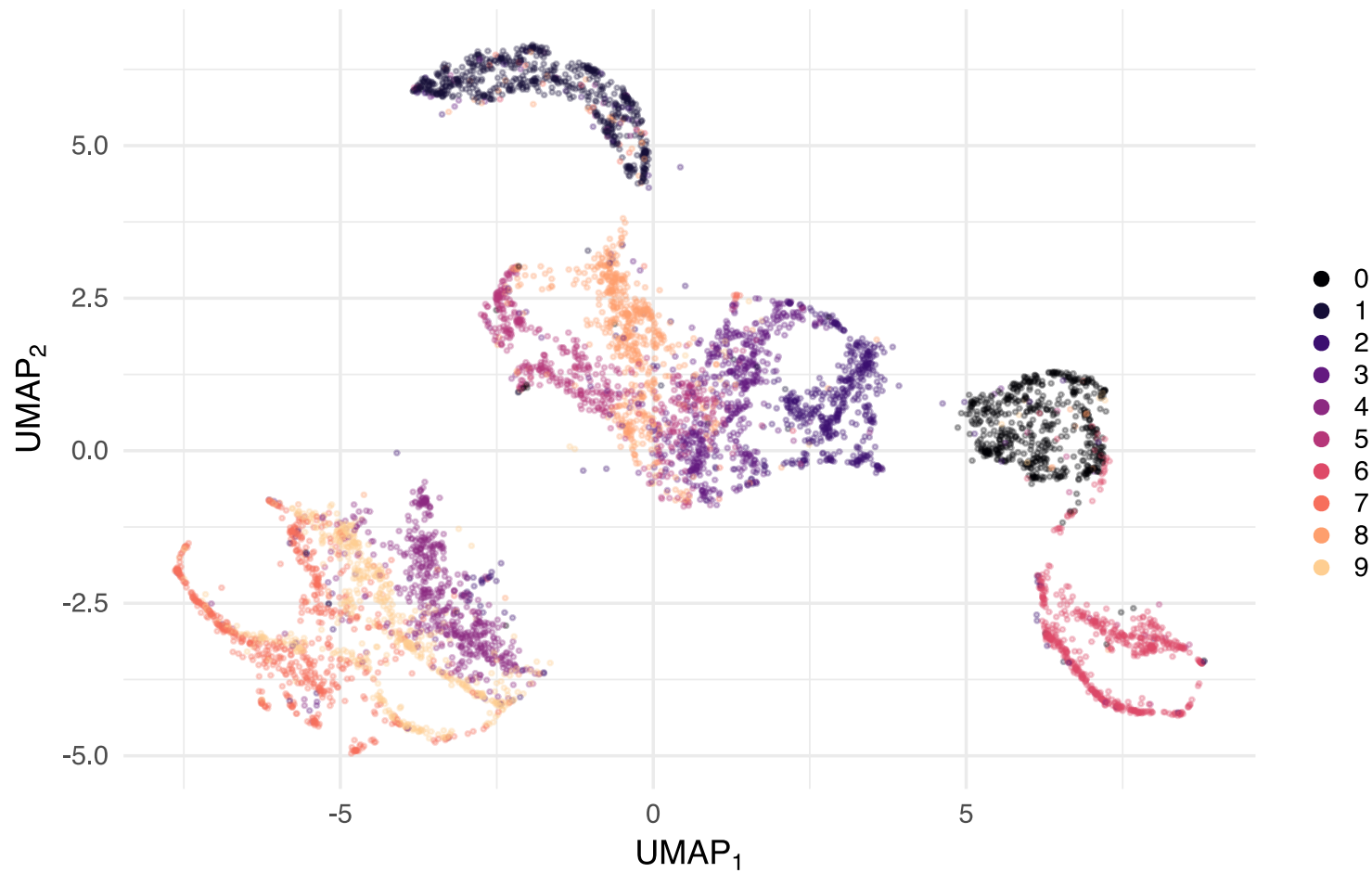
Updating our *K*-means workflow to include UMAP

1. **normalize** the pixels
2. embed the data in **2 dimensions** via UMAP
3. cluster with ***K*-means** in the embedded space

```
simple_recipe_umap =  
  recipe(  
    label ~ .,  
    data = analysis_dt ▷  
      select(label, starts_with('pixel'))  
  ) ▷  
  step_mutate(label = as.factor(label)) ▷  
  step_normalize(all_numeric_predictors()) ▷  
  step_umap(all_predictors(), num_comp = 2) ▷  
  step_rm(starts_with('pixel'))
```

So what does the UMAP embedding look like?

2-dimensional UMAP embedding of 784-dimensional MNIST pixel space



Cool. So how does this 2-D UMAP embedding perform?

Predicting digits

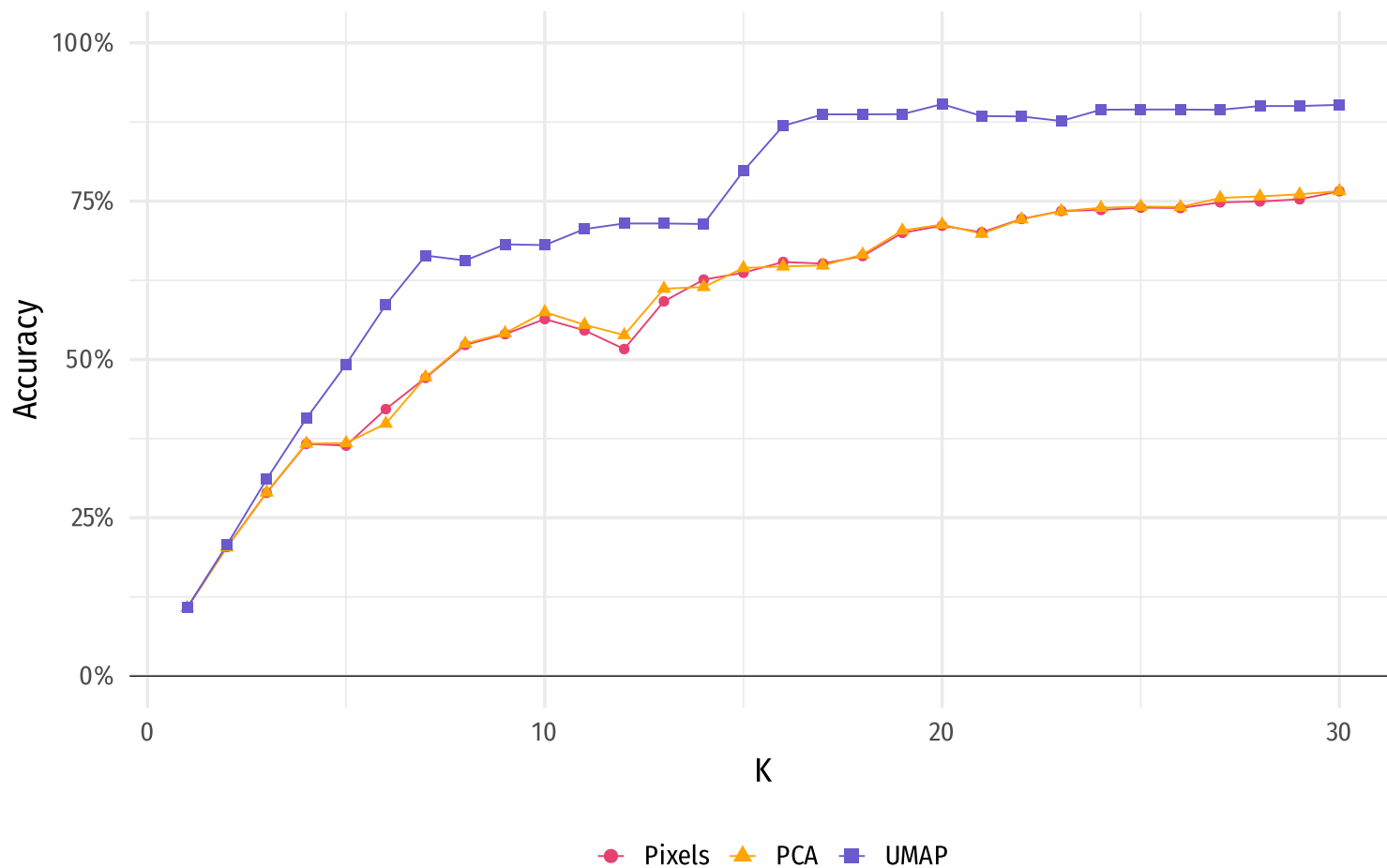
K-means in the UMAP space

With only **2 dimensions**(!!), K-means accuracy is **64.6%** (K=10).

- This result *does not* mean UMAP always wins.
- It *does* mean a **nonlinear embeddings** can preserve useful structures.
- UMAP's **nonlinearity** can capture complex relationships.

Reading handwriting likely involves such complex/nonlinear relationships.

Comparing K -means accuracy across representations and values of K



Predicting digits

Representation tradeoffs

What do these results mean?

- **Pixels** Simplest description, roughest geometry
- **PCA** Linear, compact, mathematically clean
- **UMAP** Nonlinear, visual, less interpretable

In-sample accuracy at $K = 10$

- **Pixels** 54.2%
- **PCA** 45.1%
- **UMAP** 64.6%

Representation choice can change clustering performance a lot.

Wrapping up

Wrapping up

Structure and prediction

Unsupervised tools help depicts whether the features contain signal.

- they can reduce dimension before supervised learning,
- they can reveal where classes overlap

If the goal is prediction, we usually move back to **supervised** methods.

(We could probably beat *K*-means with a more direct supervised method.)

Wrapping up

Basic image-classification intuition

Our approach today for image classification

1. learn a good **representation**,
2. use that representation to make a prediction.

Next step Neural networks start learning both pieces together.

Wrapping up

Basic image-classification intuition

Our approach today for image classification

1. learn a good **representation**,
2. use that representation to make a prediction.

Next step Neural networks start learning both pieces together.

Reminder We have been treating the **representation** as a **preprocessing step**, but it is really part of the model. Cross validation should reflect that.

Wrapping up

Final thoughts

What did we see?

- **K-means** is simple; it depends heavily on geometry.
Other clustering methods have different assumptions and objectives.
- Raw, disjoint **pixel space** is high-dimensional and awkward.
Convolutions and nonlinear embeddings can help.
- **PCA** gives a linear low-dimensional summary.
Reliance on linearity can be too restrictive.
- **UMAP** gives a nonlinear, low-dimensional summary.

Next stop Neural networks and a more serious classification pipeline.

Table of contents

Opening

- Admin
- Why digits?
- Image classification
- Supervised vs. unsupervised

Clustering

- K-means
- Choosing K

Dimension reduction

- PCA
- UMAP

Closing

- Structure and prediction
- Final thoughts